Repeated Transaction Detection

1. Problem Statement

Repeated Transaction Detection is a Python program designed to identify duplicate transaction IDs, detect transactions that exceed a specified amount, and list suspicious transactions. The program accepts transaction details from the user, including transaction IDs and transaction amounts. It compares the transaction IDs to find duplicates and checks each transaction amount against a given limit. Finally, it displays the duplicate transaction IDs, high-value transactions, and suspicious entries. This program demonstrates how Python lists, loops, conditions, and comparison operators can be used to analyze transaction records and identify potentially unusual activities.

2. Objectives

- To store transaction IDs and amounts using Python lists.
- To identify duplicate transaction IDs.
- To detect transactions that exceed a specified amount.
- To list suspicious transactions based on predefined conditions.
- To understand the use of loops, conditions, and comparison operators in Python.
- To develop basic transaction-monitoring and data-analysis skills.

3. Assumptions

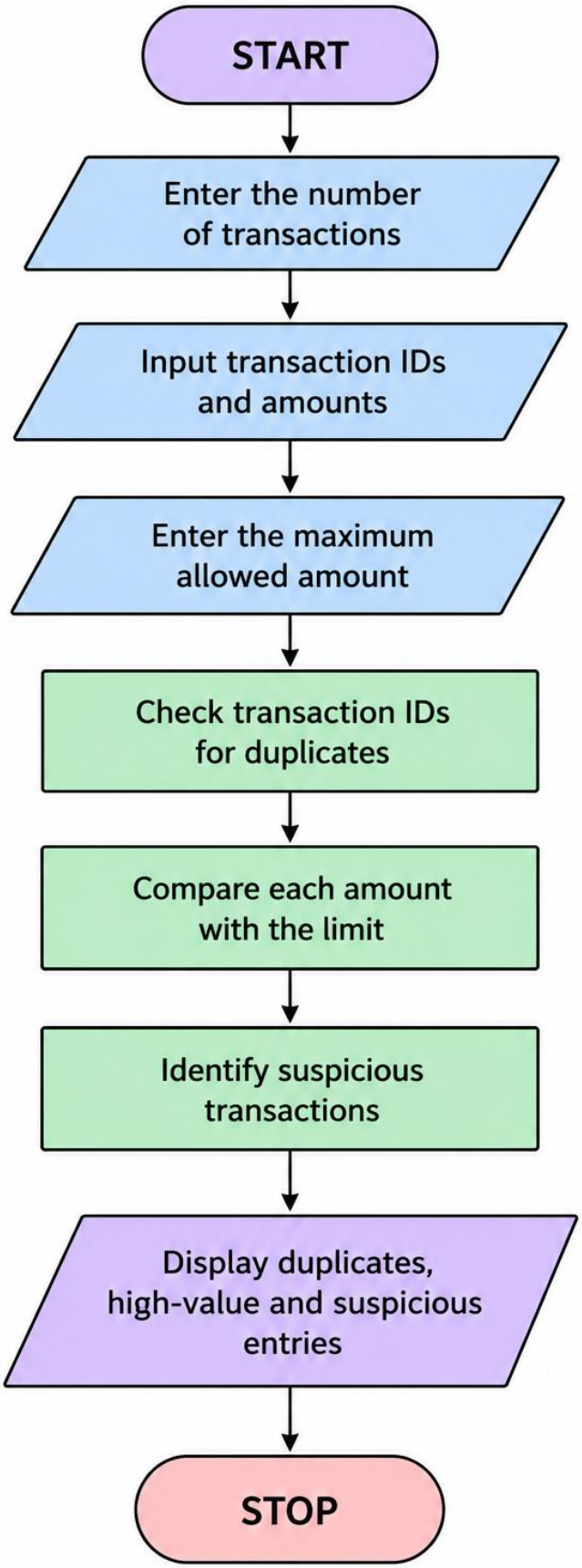
1. The user enters a fixed number of transactions.
2. Each transaction contains a transaction ID and a non-negative amount.
3. Transaction IDs are entered as text and may repeat.
4. The user provides a valid, non-negative amount limit.
5. A transaction is considered a duplicate if its ID appears more than once.
6. A transaction is considered high-value if its amount is greater than the specified limit.

7. A transaction is marked as suspicious if its ID is duplicated or its amount exceeds the limit.
8. The program identifies suspicious entries based only on these rules; it does not prove that fraud has occurred.

4. Algorithm

1. Start the program.
2. Ask the user to enter the number of transactions.
3. Create empty lists to store transaction IDs and amounts.
4. Read the ID and amount for each transaction.
5. Ask the user to enter the transaction amount limit.
6. Create an empty list to store duplicate transaction IDs.
7. Compare each transaction ID with the other IDs.
8. If an ID occurs more than once, add it to the duplicate list only once.
9. Check each transaction amount against the specified limit.
10. Display all duplicate transaction IDs.
11. Display transactions whose amounts exceed the limit.
12. Display suspicious transactions that have duplicate IDs or high amounts.
13. Stop the program.

5. Flowchart


START
Enter the number of transactions
Input transaction IDs and amounts
Enter the maximum allowed amount
Check transaction IDs for duplicates
Compare each amount with the limit
Identify suspicious transactions
Display duplicates, high-value and suspicious entries
STOP

6. Pseudocode

```
BEGIN

INPUT number_of_transactions

CREATE empty lists: transaction_ids, amounts

FOR each transaction
    INPUT transaction_id
    INPUT transaction_amount
    ADD transaction_id to transaction_ids
    ADD transaction_amount to amounts
END FOR

INPUT amount_limit

CREATE empty list duplicate_ids

FOR each transaction_id in transaction_ids
    IF transaction_id occurs more than once
       AND transaction_id is not in duplicate_ids
        ADD transaction_id to duplicate_ids
    END IF
END FOR
```

```
DISPLAY duplicate_ids

FOR each transaction
   IF amount is greater than amount_limit
      DISPLAY transaction as high-value
   END IF
END FOR

FOR each transaction
   IF transaction ID is duplicated
      OR amount is greater than amount_limit
      DISPLAY transaction as suspicious
   END IF
END FOR

END
```

7. Python Actual Code

```
transaction_ids = []
amounts = []

n = int(input("Enter the number of transactions: "))

# Input transaction details
```

```
for i in range(n):
    print("\nTransaction", i + 1)

    tid = input("Enter transaction ID: ")
    amount = float(input("Enter transaction amount: "))

    transaction_ids.append(tid)
    amounts.append(amount)

# Input the amount limit
limit = float(input("\nEnter the amount limit: "))

# Find duplicate transaction IDs
duplicate_ids = []

for tid in transaction_ids:
    if transaction_ids.count(tid) > 1:
        if tid not in duplicate_ids:
            duplicate_ids.append(tid)

print("\n--- Duplicate Transaction IDs ---")

if len(duplicate_ids) == 0:
    print("No duplicate transaction IDs found.")
```

```
else:
    for tid in duplicate_ids:
        print(tid)

# Find high-value transactions
print("\n--- High-Value Transactions ---")

found_high = False

for i in range(n):
    if amounts[i] > limit:
        print("ID:", transaction_ids[i],
              "| Amount: Rs.", amounts[i])
        found_high = True

if not found_high:
    print("No high-value transactions found.")

# Find suspicious transactions
print("\n--- Suspicious Transactions ---")

found_suspicious = False

for i in range(n):
```

```
    if (transaction_ids[i] in duplicate_ids
            or amounts[i] > limit):
        print("ID:", transaction_ids[i],
              "| Amount: Rs.", amounts[i])
        found_suspicious = True

if not found_suspicious:
    print("No suspicious transactions found.")
```

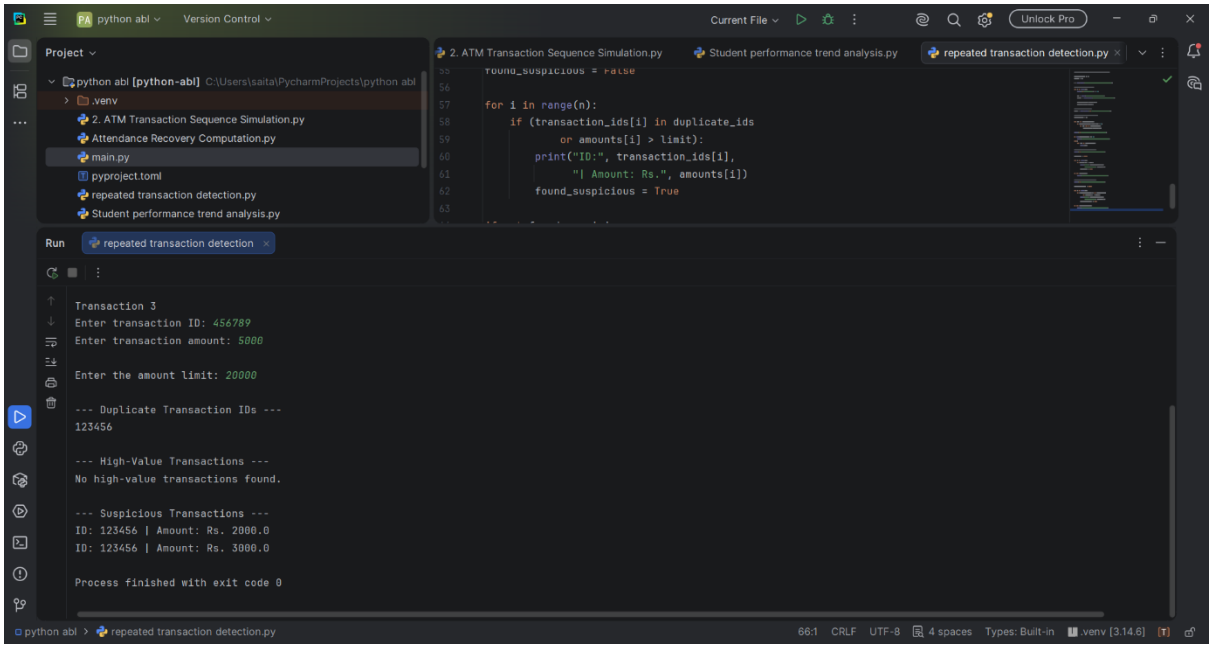
8. Explanation of the Code

Code section	Explanation
transaction_ids = []	Creates a list to store transaction IDs.
amounts = []	Creates a list to store transaction amounts.
input()	Accepts transaction details from the user.
for i in range(n)	Repeats the input process for all transactions.
append()	Adds transaction details to their respective lists.
count(tid)	Counts how many times a transaction ID appears.
if tid not in duplicate_ids	Prevents the same duplicate ID from being displayed repeatedly in the duplicate list.
amounts[i] > limit	Checks whether a transaction exceeds the amount limit.
or operator	Identifies a transaction as suspicious if its ID is duplicated or its amount exceeds the limit.
found_high and found_suspicious	Track whether any matching transactions were found.
print()	Displays the results on the screen.

Python concepts used

- Lists: Store transaction IDs, amounts, and duplicate IDs.
- Loops: Process multiple transactions.
- Conditions: Check for duplicates and high-value transactions.
- Comparison operators: Compare transaction counts and amounts.
- Logical operators: Combine suspicious-transaction conditions.

9. Output



10. Conclusion

The Repeated Transaction Detection program demonstrates how Python can be used to analyze transaction records and identify potentially suspicious activities. It detects duplicate transaction IDs, finds transactions that exceed a specified amount, and displays suspicious entries based on these conditions. Through this program, we learn to use lists, loops, conditions, comparison operators, and logical operators. This case study provides a basic understanding of transaction monitoring and data analysis. It can be further improved by adding dates, transaction locations, and more advanced fraud-detection rules.